



## EXPERIMENT-2

**AIM: Design a neural network for classifying movie reviews (Binary Classification) using IMDB (Internet Movie Data Base) dataset.**

**DESCRIPTION:**

We'll be working with "IMDB dataset", a set of 50,000 highly-polarized reviews from the Internet Movie Database. They are split into 25,000 reviews for training and 25,000 reviews for testing, each set consisting in 50% negative and 50% positive reviews. You should never test a machine learning model on the same data that you used to train it! Just because a model performs well on its training data doesn't mean that it will perform well on data it has never seen, and what you actually care about is your model's performance on new data. Just like the MNIST dataset, the IMDB dataset comes packaged with Keras. It has already been preprocessed: the reviews (sequences of words) have been turned into sequences of integers, where each integer stands for a specific word in a dictionary.

**PROGRAM:**

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import models
from tensorflow.keras import layers
from tensorflow.keras.datasets import imdb

# Build the model
model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

# Load the dataset
(X_train, y_train), (X_test, y_test) = imdb.load_data()

# Combine train and test data
X = np.concatenate((X_train, X_test), axis=0)
y = np.concatenate((y_train, y_test), axis=0)

# Summarize size
print("Training data:")
print(X.shape)
print(y.shape)

# Summarize number of classes
print("Classes:")
print(np.unique(y))
```

[illegible]



```
# Summarize number of words
print("Number of unique words:")
print(len(np.unique(np.hstack(X))))

# Summarize review length
print("Review length:")
result = [len(x) for x in X]
print("Mean %.2f words (Std: %.2f)" % (np.mean(result), np.std(result)))

# Plot review length
plt.boxplot(result)
plt.title("Review Length Distribution")
plt.ylabel("Number of Words")
plt.show()
```

**OUTPUT:**

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>  
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not  
pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer  
using an `Input(shape)` object as the first layer in the model instead.

```
super().init(activity_regularizer=activity_regularizer,**kwargs)
```

**17464789/17464789 ————— 1s 0us/step**

Training data:

(50000,)

(50000,)

**Classes:**

$$[0 \ 1]$$

Number of unique words:

88585

Review length:

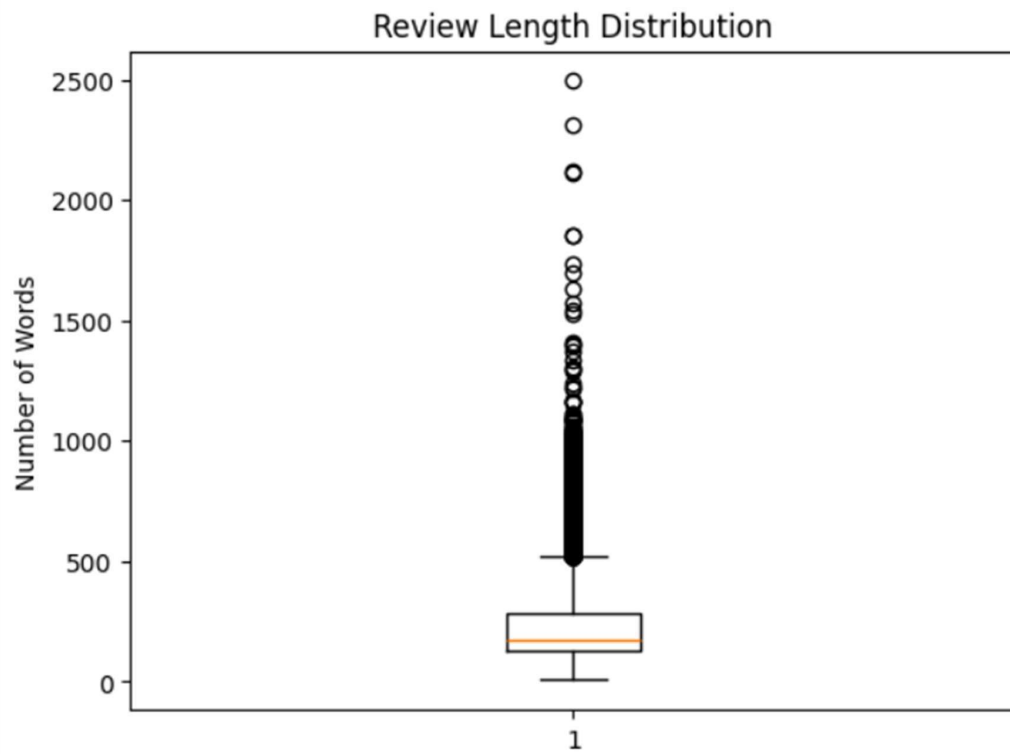
Mean 234.76 words (Std: 172.91)

[illegible]

Date :



Page No. :

[illegible]



## EXPERIMENT-5

## AIM: Build a Convolution Neural Network for MNIST Hand written Digit Classification.

**DESCRIPTION:**

The MNIST handwritten digit classification problem is a standard dataset used in computer vision and deep learning. Although the dataset is effectively solved, it can be used as the basis for learning and practicing how to develop, evaluate, and use convolutional deep learning neural networks for image classification from scratch. This includes how to develop a robust test harness for estimating the performance of the model, how to explore improvements to the model, and how to save the model and later load it to make predictions on new data.

- How to develop a test harness to develop a robust evaluation of a model and establish a baseline of performance for a classification task.
- How to explore extensions to a baseline model to improve learning and model capacity.
- How to develop a finalized model, evaluate the performance of the final model, and use it to make predictions on new images.

**PROGRAM:**

```
import tensorflow as tf

from tensorflow.keras.datasets import mnist

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

from tensorflow.keras.utils import to_categorical

import numpy as np

import matplotlib.pyplot as plt


# 1. Load the MNIST data

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()


# 2. Preprocess the Data

# Reshape images to (batch_size, height, width, channels)

train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255

test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255
```

[illegible]

Date :



Page No. :

```
# Convert labels to one-hot encoding
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)

# 3. Build the CNN Model
model = Sequential()

# First Conv block
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D((2, 2)))

# Second Conv block
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))

# Flatten layer
model.add(Flatten())

# Fully connected layers
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5)) # Dropout for regularization
model.add(Dense(10, activation='softmax')) # Output layer for 10 classes

# 4. Compile the Model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

[illegible]

Date :



Page No. :

```
# Display model summary
model.summary()

# 5. Train the Model

history = model.fit(
    train_images,
    train_labels,
    epochs=5,
    batch_size=64,
    validation_data=(test_images, test_labels)
)

# 6. Evaluate the Model

test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print(f"\nTest accuracy: {test_acc * 100:.2f}%")

# 7. Visualize Results

plt.figure(figsize=(12, 4))

# Accuracy Plot

plt.subplot(1, 2, 1)

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss Plot

plt.subplot(1, 2, 2)
```



```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
```

```
plt.tight_layout()
```

plt.show()

**OUTPUT:**

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 1s 0us/step

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

| Layer (type)                   | Output Shape       | Param # |
|--------------------------------|--------------------|---------|
| conv2d (Conv2D)                | (None, 26, 26, 32) | 320     |
| max_pooling2d (MaxPooling2D)   | (None, 13, 13, 32) | 0       |
| conv2d_1 (Conv2D)              | (None, 11, 11, 64) | 18,496  |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64)   | 0       |
| flatten (Flatten)              | (None, 1600)       | 0       |
| dense (Dense)                  | (None, 128)        | 204,928 |
| dropout (Dropout)              | (None, 128)        | 0       |
| dense_1 (Dense)                | (None, 10)         | 1,290   |

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)



Epoch 1/5

938/938 ————— 57s 58ms/step - accuracy: 0.8408 - loss: 0.4947 - val\_accuracy: 0.9842 - val\_loss: 0.0484

Epoch 2/5

938/938 ————— 52s 55ms/step - accuracy: 0.9743 - loss: 0.0853 - val\_accuracy: 0.9879 - val\_loss: 0.0347

Epoch 3/5

938/938 ————— 50s 54ms/step - accuracy: 0.9810 - loss: 0.0609 - val\_accuracy: 0.9900 - val\_loss: 0.0302

Epoch 4/5

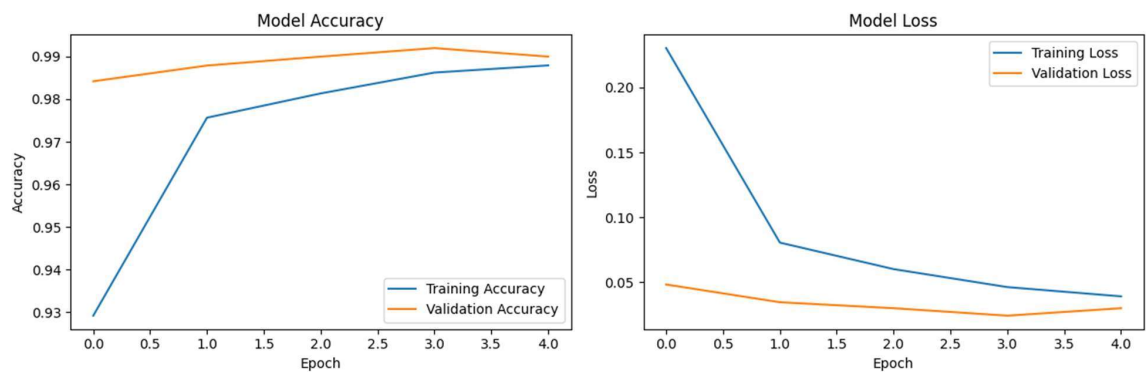
938/938 ————— 83s 55ms/step - accuracy: 0.9863 - loss: 0.0472 - val\_accuracy: 0.9920 - val\_loss: 0.0244

Epoch 5/5

938/938 ————— 51s 55ms/step - accuracy: 0.9878 - loss: 0.0400 - val accuracy: 0.9900 - val loss: 0.0302

313/313 - 3s - 8ms/step - accuracy: 0.9900 - loss: 0.0302

**Test accuracy: 99.00%**





## EXPERIMENT-8

**AIM: : Implement one hot encoding of words or characters.**

**DESCRIPTION:**

One-hot encoding is the most common, most basic way to turn a token into a vector. You already saw it in action in our initial IMDB and Reuters examples from chapter 3 (done with words, in our case). It consists in associating a unique integer index to every word, then turning this integer index  $i$  into a binary vector of size  $N$ , the size of the vocabulary, that would be all-zeros except for the  $i$ -th entry, which would be 1.

Of course, one-hot encoding can be done at the character level as well. To unambiguously drive home what one-hot encoding is and how to implement it, here are two toy examples of one-hot encoding: one for words, the other for characters.

**PROGRAM:**

```
import numpy as np

# Sample sentences

samples = {
    'Jupiter has 79 known moons .',
    'Neptune has 14 confirmed moons !'
}

# Step 1: Create token index (vocabulary)

token_index = {}

counter = 0

for sample in samples:
    for considered_word in sample.split():
        if considered_word not in token_index:
            counter += 1
            token_index[considered_word] = counter

print("Token Index:")
```

[illegible]



```
print(token_index)
```

## # Step 2: Set maximum sentence length

max\_length = 6

```
# Step 3: Create zero tensor (samples × max_length × vocabulary_size+1)
```

```
results = np.zeros(
    (len(samples), max_length, max(token_index.values()) + 1)
)
```

### # Step 4: One-hot encoding

```
for i, sample in enumerate(samples):
```

```
for j, considered_word in enumerate(sample.split()):
```

```
if j < max_length:
```

```
index = token_index[considered_word]
```

```
results[i, j, index] = 1
```

### # Step 5: Display results

```
print("\nOne-Hot Encoded Tensor:")
```

```
print(results)
```

**OUTPUT:**

Token Index:

{'Neptune': 1, 'has': 2, '14': 3, 'confirmed': 4, 'moons': 5, '!': 6, 'Jupiter': 7, '79': 8, 'known': 9, '.': 10}

### One-Hot Encoded Tensor:

[[[0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]

[0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]

[0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]

[0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]

[0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]

[0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]

Date :



```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]]]
```

[illegible]



## EXPERIMENT-10

**AIM: Implement a Recurrent Neural Network for IMDB movie review classification problem.**

**DESCRIPTION:**

The process of building a Sentiment Analysis model using Recurrent Neural Networks (RNNs) to classify movie reviews as positive or negative. Sentiment analysis is a popular Natural Language Processing (NLP) task that involves determining the sentiment or emotion expressed in a piece of text. We will use Python, TensorFlow, and Keras to implement the RNN model and analyze the results.

Recurrent Neural Networks (RNN) are to the rescue when the sequence of information is needed to be captured (another use case may include Time Series, next word prediction, etc.). Due to its internal memory factor, it remembers past sequences along with current input which makes it capable to capture context rather than just individual words.

The IMDB dataset consists of movie reviews from the IMDb website, along with labels indicating whether each review is “positive” or “negative” based on the reviewer’s opinion. It is designed for binary sentiment classification, with 25,000 reviews for training and an additional 25,000 for testing. Both the training and testing sets have an equal number of positive and negative reviews, making them balanced for sentiment analysis tasks.

**PROGRAM:**

```
import numpy as np
```

```
import pandas as pd
```

```
import tensorflow as tf
```

```
import re
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import pickle
```

```
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding
```

```
from tensorflow.keras.datasets import imdb
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.preprocessing import sequence
```

```
# Load IMDB dataset (top 5000 frequent words only)
```



```
vocab_size = 5000

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

print(x_train[0]) # Print first encoded review

# Get word index dictionary
word_idx = imdb.get_word_index()

# Reverse dictionary (number → word)
word_idx = {i: word for word, i in word_idx.items()}

# Print decoded review
print([word_idx.get(i, '?') for i in x_train[0]])

# Print max and min review length
print("Max length of a review:: ",
      len(max((x_train.tolist() + x_test.tolist()), key=len)))

print("Min length of a review:: ",
      len(min((x_train.tolist() + x_test.tolist()), key=len)))

# Pad all reviews to 400 words
max_words = 400

x_train = sequence.pad_sequences(x_train, maxlen=max_words)
x_test = sequence.pad_sequences(x_test, maxlen=max_words)

# Create validation split
x_valid, y_valid = x_train[:64], y_train[:64]
x_train, y_train = x_train[64:], y_train[64:]
```



```
# Set embedding size
embd_len = 32

# Build Simple RNN model
RNN_model = Sequential(name="Simple_RNN")

# Embedding layer (word → vector)
RNN_model.add(Embedding(vocab_size, embd_len, input_length=max_words))

# RNN layer
RNN_model.add(SimpleRNN(128, activation='tanh'))

# Output layer (binary classification)
RNN_model.add(Dense(1, activation='sigmoid'))

# Show model summary
print(RNN_model.summary())

# Compile model
RNN_model.compile(
    loss="binary_crossentropy",
    optimizer='adam',
    metrics=['accuracy']
)

# Train model
history = RNN_model.fit(
    x_train_,
    y_train_,
    batch_size=64,
```



```
epochs=5,
verbose=1,
validation_data=(x_valid, y_valid)
)

# Evaluate on test data
print()
print("Simple_RNN Score---> ",
      RNN_model.evaluate(x_test, y_test, verbose=0))

# Plot accuracy graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

**OUTPUT:**

KDownloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>

17464789/17464789 ————— 0s 0us/step

Downloading data from [https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb\\_word\\_index.json](https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json)

1641221/1641221 ————— 0s 0us/step

Max length of a review:: 2494

Min length of a review:: 7

Model: "Simple\_RNN"

[illegible]

Date :



Page No. :

| Layer (type)           | Output Shape | Param #     |
|------------------------|--------------|-------------|
| embedding (Embedding)  | ?            | 0 (unbuilt) |
| simple_rnn (SimpleRNN) | ?            | 0 (unbuilt) |
| dense (Dense)          | ?            | 0 (unbuilt) |

**Total params:** 0 (0.00 B)  
**Trainable params:** 0 (0.00 B)  
**Non-trainable params:** 0 (0.00 B)

None

Epoch 1/5

```
390/390 ————— 71s 177ms/step - accuracy: 0.5522 - loss: 0.6776 -  
val accuracy: 0.6094 - val loss: 0.6216
```

Epoch 2/5

```
390/390 ————— 68s 176ms/step - accuracy: 0.7390 - loss: 0.5211 -  
val_accuracy: 0.8438 - val_loss: 0.4199
```

Epoch 3/5

```
390/390 ————— 68s 173ms/step - accuracy: 0.8286 - loss: 0.3975 -  
val accuracy: 0.7500 - val loss: 0.5283
```

Epoch 4/5

390/390 — 84s 179ms/step - accuracy: 0.7831 - loss: 0.4585 -  
val accuracy: 0.6562 - val loss: 0.5939

Epoch 5/5

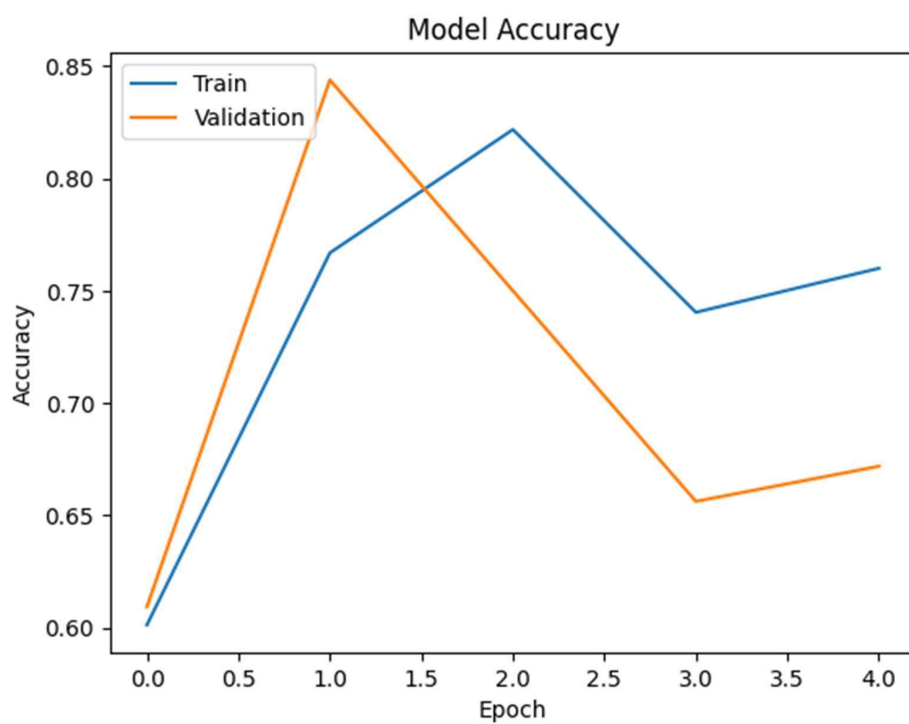
```
390/390 — 80s 174ms/step - accuracy: 0.7615 - loss: 0.4889 -  
val accuracy: 0.6719 - val loss: 0.6045
```

Simple RNN Score---> [0.5558714270591736, 0.7087600231170654]

Date :



Page No. :

[illegible]